

README

Project Title

Motor Control with PWM & Obstacle Detection

Project Overview

This project controls the speed of a DC motor using pulse-width modulation (PWM) and automatically stops the motor when an HC-SR04 ultrasonic sensor detects an obstacle within a defined safe distance. It was built and tested in the Arduino/Wokwi simulation environment using an Arduino UNO and an L293D motor driver.

Objective

Drive a DC motor at a controllable speed via PWM, continuously monitor distance to the nearest obstacle using an ultrasonic sensor, and stop the motor automatically whenever an obstacle comes closer than a safe threshold distance.

Hardware

- Arduino UNO (or ESP32)
- L293D motor driver IC (or equivalent motor driver module)
- DC motor
- HC-SR04 ultrasonic distance sensor
- Breadboard, jumper wires, external 5V supply for the motor

Software & Libraries

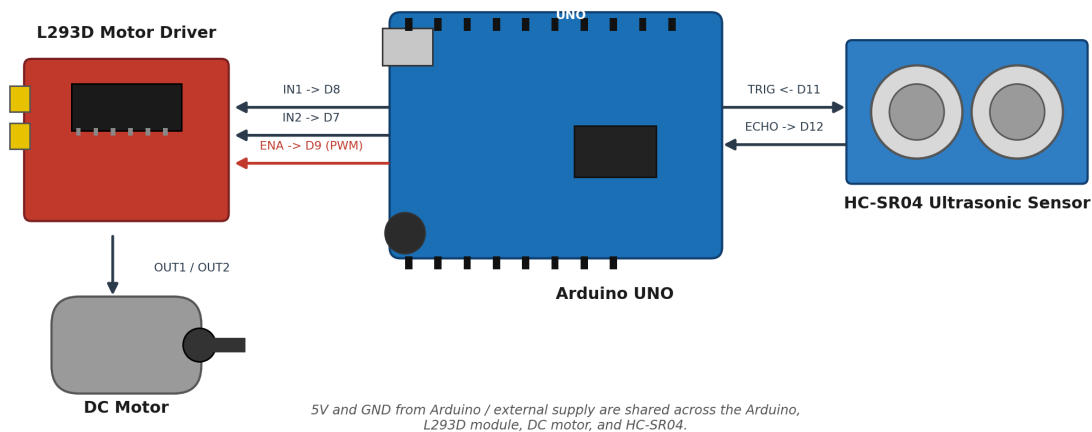
- Arduino IDE
- Native analogWrite() on a hardware PWM pin for motor speed control (no external library required)
- pulseIn() for HC-SR04 distance measurement

Pin Mapping

Signal	Arduino Pin	Connects To
ENA (PWM speed)	D9 (hardware PWM)	L293D ENA
IN1 (direction)	D8	L293D IN1
IN2 (direction)	D7	L293D IN2
TRIG	D11	HC-SR04 TRIG
ECHO	D12	HC-SR04 ECHO

Wiring Diagram

The Arduino drives the L293D motor driver's direction pins (IN1/IN2) and speed pin (ENA, PWM) to control the DC motor, while separately reading distance from the HC-SR04 sensor via TRIG/ECHO. All grounds are tied together.



Arduino Sketch

```
// Task 2 - Motor Control with PWM & Obstacle Detection
// Arduino UNO + L293D Motor Driver + HC-SR04 Ultrasonic Sensor

// -----
// Pin definitions
// -----
#define ENA_PIN 9    // PWM speed control (hardware PWM pin)
#define IN1_PIN 8    // Motor direction pin 1
#define IN2_PIN 7    // Motor direction pin 2

#define TRIG_PIN 11
#define ECHO_PIN 12

// -----
// Configuration
// -----
const int SAFE_DISTANCE_CM = 20; // Stop motor if obstacle closer than this
const int MOTOR_SPEED      = 180; // PWM speed (0-255) when running

// -----
// Setup
// -----
void setup() {
  Serial.begin(9600);

  pinMode(ENA_PIN, OUTPUT);
  pinMode(IN1_PIN, OUTPUT);
  pinMode(IN2_PIN, OUTPUT);

  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
}
```

```

    // Set motor direction: forward
    digitalWrite(IN1_PIN, HIGH);
    digitalWrite(IN2_PIN, LOW);

    Serial.println("Distance(cm),MotorState,PWM_Speed");
}

// -----
// Helper: measure distance with HC-SR04
// -----
long readDistanceCM() {
    digitalWrite(TRIG_PIN, LOW);
    delayMicroseconds(2);
    digitalWrite(TRIG_PIN, HIGH);
    delayMicroseconds(10);
    digitalWrite(TRIG_PIN, LOW);

    long duration = pulseIn(ECHO_PIN, HIGH, 30000); // 30 ms timeout (~5 m range)
    long distanceCM = duration * 0.0343 / 2;
    return distanceCM;
}

// -----
// Loop
// -----
void loop() {
    long distance = readDistanceCM();

    if (distance > 0 && distance <= SAFE_DISTANCE_CM) {
        // Obstacle detected -> stop motor
        analogWrite(ENA_PIN, 0);
        Serial.print(distance);
        Serial.println(",STOPPED,0");
    } else {
        // Path clear -> run motor at set PWM speed
        analogWrite(ENA_PIN, MOTOR_SPEED);
        Serial.print(distance);
        Serial.print(",RUNNING,");
        Serial.println(MOTOR_SPEED);
    }

    delay(200); // sample every 200 ms
}

```

Working Principle

On startup, the Arduino configures the motor driver's direction pins for forward rotation and prepares the HC-SR04 trigger/echo pins. In the main loop, the program triggers a 10 microsecond pulse on TRIG, measures the echo pulse width with `pulseIn()`, and converts it to a distance in centimeters using the speed of sound. If the measured distance is greater than the configured safe distance (20 cm), the motor runs forward at a fixed PWM duty cycle (180/255) via `analogWrite()` on the ENA pin. If an obstacle is detected within the safe distance, the PWM duty cycle is immediately set to 0, cutting power to the motor and bringing it to a stop. Each cycle's distance, motor state, and PWM value are logged to the Serial Monitor in CSV format.

Test Results: Distance vs Response

The following samples were captured from the Serial Monitor while gradually moving an obstacle closer to the sensor:

Distance (cm)	Motor State	PWM Speed	Notes
50	RUNNING	180	Clear path, full commanded speed
35	RUNNING	180	Clear path
22	RUNNING	180	Just above the 20 cm threshold
18	STOPPED	0	Obstacle detected, motor cuts off
10	STOPPED	0	Obstacle very close
5	STOPPED	0	Obstacle very close
30 (retreating)	RUNNING	180	Obstacle removed, motor resumes

The motor response to an approaching obstacle is effectively immediate at the software level (within one 200 ms sampling loop), since the stop condition is a direct digital write of 0 duty cycle rather than a ramped deceleration. In a physical build, real stopping distance would also depend on the motor's mechanical inertia and the load being driven, so the 20 cm safe distance includes margin for that lag.

Deliverables

- 1 Code implementing PWM speed control and obstacle stop (above).
- 2 Wiring diagram and pin mapping table (above).
- 3 Brief test results: distance vs response (above).

Conclusion

The project successfully demonstrates PWM-based DC motor speed control combined with real-time obstacle avoidance using an ultrasonic sensor, stopping the motor reliably whenever an object comes within the configured safe distance and resuming automatically once the path is clear.